

Five architectural forks, now decided

Scope	Token-driven design system and Angular component library, three waves, solo maintainer
Constraints	Angular 20 through current and forward; standalone only; zero hard-coded values — every property resolves to a CSS custom property
Status	All six decisions settled. Read for approval; the build plan follows from them unchanged.

Raising the floor to Angular 20 is the right call and it improves the system materially — signal inputs, signal queries and a zoneless-ready library, none of which you could have had on a 16 baseline. It also *simplifies* every fork downstream, because you are no longer designing around an API you cannot use.

DECISIONS OF RECORD

- | | |
|--------------------------|--|
| 01 · Versions | Angular 20 → current. One package, compiled on the 20 floor. Signals throughout, zero NgModules. |
| 02 · Distribution | Native Angular standalone components via ng-packagr. No custom elements. |
| 03 · Tokens | Hand-authored DTCG JSON → Style Dictionary → CSS custom properties + TS types. |
| 04 · Theming | One sheet; mode and density as independent attribute axes; a thin directive applies them. |
| 05 · Overrides | Emulated encapsulation, variants as data- attributes, component tokens as the public override API. |
| 06 · Scope | Three waves. Table / Data Grid explicitly out of v1. |

Angular 20 as the floor — and what that obliges you to

One mechanism still governs the version story, and it is worth restating because it is what makes "20 through current and forward" a safe promise rather than a hopeful one: a library published in **partial-ivy** form — what `ng-packagr` emits by default — is compiled by each consuming application's own compiler. Forward compatibility is guaranteed, backward compatibility is not. So **build the library workspace on your floor** — Angular 20 — and it installs cleanly into 21 and beyond.

What the raised floor changes is the API you get to author with. Four things are now available that the 16 baseline would have denied you, and one of them you could not have retrofitted later at any reasonable price.

Signal inputs and outputs

`input()`, `input.required()`, `output()` and `model()` replace every decorator. Required inputs become a compile-time guarantee instead of a runtime null check, and `model()` gives you two-way binding on Switch and Tabs without writing a single `EventEmitter`.

Signal queries

`viewChild()` and `contentChildren()` are reactive values, not lifecycle-timed assignments. This matters most for exactly the components in wave 3 — Tabs, Menu and Select all need to observe a changing set of children, which is awkward with `QueryList` and trivial with a signal.

Zoneless readiness — the one you cannot retrofit

Zoneless change detection is stable from 20. A library whose reactivity runs entirely through signals works in zoneless *and* zone-based applications; a library that quietly depends on zone patching works only in the latter, and finding out later means auditing every component. Authoring zoneless-safe from the first commit costs nothing. Discovering you did not is a rewrite.

Modern authoring ergonomics

The host metadata object instead of scattered @HostBindings, built-in control flow in templates, and linkedSignal for state that derives from an input but stays writable — a Tabs selected index that resets when the tab set changes is precisely that shape.

So the canonical component becomes this — and note there is no standalone: true, because from Angular 19 standalone is the default and stating it is now noise:

```
@Component({
  selector: 'button[dsButton]',
  template: '<ng-content />',
  styleUrls: ['./button.css'],
  changeDetection: ChangeDetectionStrategy.OnPush,
  host: {
    '[attr.data-tone]': 'tone()',
    '[attr.data-size]': 'size()',
    '[attr.aria-busy]': 'loading() || null',
    '[disabled]': 'disabled() || loading()',
  },
})
export class DsButton {
  readonly tone = input<DsTone>('neutral');
  readonly size = input<DsSize>('md');
  readonly loading = input(false, { transform: booleanAttribute });
  readonly disabled = input(false, { transform: booleanAttribute });
}
```

Two conventions visible there, worth adopting library-wide: DsButton rather than DsButtonComponent, following Angular's current style guide, which drops the suffix; and DsTone / DsSize generated by the token pipeline, so deleting a size token is a compile error rather than a silent visual regression.

DECISION, WITH THREE OBLIGATIONS ATTACHED

One package, workspace on Angular 20, peer range "@angular/core": ">=20.0.0 <22.0.0". In exchange:

1. **Run the sandbox zoneless from day one.** Add `provideZonelessChangeDetection()` before you write `Button`. No `setTimeout`-driven view updates, no `NgZone.onStable`, nothing that assumes zone patching.
2. **No NgModules at all** — not even one convenience re-export. That is how a standalone-only library quietly stops being one.
3. **Treat the peer range as a support promise.** Widen it deliberately, on a commit that ran CI against the new major — never as a convenience edit.

What it costs: honestly, very little — which is why this was the right change. Your CI matrix is now two jobs (install into 20, install into current) rather than the six a 16 floor implied, and you will not spend the project's first month writing a 2023-era API you intend to migrate.

One caveat to carry into wave 2: signals have *not* replaced `ControlValueAccessor`. It remains the integration point for Angular forms in 20, so `Input`, `Checkbox`, `Radio` and `Switch` still implement it. Hold the value in a signal internally and let the CVA methods write to it — that way the component is signal-native and still works with `FormControl` and `ngModel`.

FORK 02 CONFIRMED

Angular components only — no custom elements

With no framework boundary to cross, the tradeoff is lopsided. Custom elements solve exactly one problem, and it is not one you have. Keeping this for the record:

CAPABILITY	ANGULAR LIBRARY	CUSTOM ELEMENTS
Type-checked bindings, signal inputs	Yes	No — attributes are strings
Content projection	Yes	Shadow slots only, awkwardly
Forms — <code>ControlValueAccessor</code>	Native	Hand-written per component
Dependency injection, CDK overlays	Yes	Needs an injector bridge
Works in React / Vue / plain HTML	No	Yes

The escape hatch stays open and costs nothing to keep: because the tokens are pure CSS with no framework in them, a marketing site, an email template or a native shell can wear your identity without wearing your components. If a non-Angular surface ever does need real components, wrap the few it needs with `createCustomElement` as a second build target. Designing for that up front would have cost you the whole API.

FORK 03 **APPROVED**

DTCG JSON in the repo, compiled by Style Dictionary

Hand-authored JSON as the source of truth, emitting CSS custom properties, TypeScript types and a JSON manifest for the docs site. Git gives you review, history and CI for nothing, and there is no sync to own. Because you author the DTCG format from day one, adopting a Figma Variables pipeline later is a change of *input*, not a migration — so defer it until the token set has stopped moving and someone has volunteered to own the round-trip.

SCSS maps are ruled out on your own constraint: they resolve at build time, which puts literal values back in the output and makes runtime theming impossible.

Three tiers, with references pointing one direction only — **down**. Component tokens may read semantic tokens; semantic tokens may read primitives; nothing reads upward, and application code never touches

a primitive.

primitive	--ds-blue-600	raw value, no meaning	→ semantic tier only
semantic	--ds-color-action	role and intent	→ components + apps
component	--ds-button-bg	the override API	→ consumers may set

Name by role, never by appearance: `--ds-color-action` survives a rebrand, `--ds-color-blue` becomes a lie the day the brand goes green.

THE SETTING THAT MATTERS MOST IN YOUR ENTIRE BUILD

```
files: [{  
  destination: 'tokens.css',  
  format: 'css/variables',  
  options: { selector: ':root, :host', outputReferences: true }  
}]
```

Without `outputReferences: true`, `var(--ds-blue-600)` is flattened to a hex value in the output and runtime theming becomes impossible — teams usually discover this the week dark mode is promised. Emitting to `:root`, `:host` costs nothing and means the tokens still resolve if you ever adopt shadow DOM.

Enforce "no hard-coded values" with a machine, not a habit. A Stylelint rule that fails CI on any literal colour or raw pixel value inside `libs/ui` is what keeps the constraint true in month nine. Working solo you have no reviewer to catch a slip, so the linter *is* your reviewer.

FORK 04 APPROVED

Mode and density as independent attribute axes

Your three theming requirements are not one list. Light/dark and density are **independent axes** on the same mechanism; per-component override is a different mechanism entirely, and it is Fork 05. Treat them as one and you get a combinatorial explosion — two modes × two densities is already four built stylesheets, and a brand axis would make eight.

Each axis is an attribute that re-points semantic tokens. Primitives never change. They compose without multiplying, they nest — a dark panel inside a light page is free — and switching is a single attribute write with no rebuild and no flash.

```
/* Axis 1 – mode. Re-points semantic tokens only. */
[data-ds-mode="light"] { --ds-color-surface: var(--ds-neutral-0);
                        --ds-color-text: var(--ds-neutral-900); }
[data-ds-mode="dark"]  { --ds-color-surface: var(--ds-neutral-900);
                        --ds-color-text: var(--ds-neutral-50); }

/* Axis 2 – density. Independent of mode, so the two compose. */
[data-ds-density="comfortable"] { --ds-control-height: 40px; }
[data-ds-density="compact"]     { --ds-control-height: 32px; }
```

Your phase 4 provider is then genuinely small — and on the 20 baseline it gets smaller still, since signal inputs collapse the host bindings into the decorator. It must not know what a colour is:

```
@Directive({
  selector: '[dsTheme]',
  host: {
    '[attr.data-ds-mode]': 'mode()',
    '[attr.data-ds-density]': 'density()',
  },
})
export class DsTheme {
  readonly mode = input<DsMode>('light');
  readonly density = input<DsDensity>('comfortable');
}
```

Pair it with a tiny root-level service holding the preference as a signal and persisting it — that is the whole theming runtime. Rejected for the record: a stylesheet per theme (cannot nest, cannot compose, doubles output per axis), and theme objects injected through DI and applied as inline styles (bypasses the cascade, so consumers can override nothing).

One guarantee to build in during phase 1: pair every surface token with an -on- foreground token, and contrast-test the pairs in CI. That is what stops a future theme from producing illegible combinations you have no way to catch by eye.

FORK 05 APPROVED

Component tokens are the whole override API

Every design system eventually meets a team that needs a button you did not design. If you have not said how, they will use `::ng-deep` and a specificity war, and your next release will break their page.

Encapsulation: keep `ViewEncapsulation.Emulated`, the default. Shadow DOM sounds like the principled choice and brings real problems — form association, focus delegation, global font loading — plus a second API surface in `::part` that you would have to design and version. Emulated encapsulation plus variants expressed as `data-` attributes already gives you effective isolation, and a consumer's stray class can never collide with an attribute selector.

Supported, in order of preference — set a documented component token on the host; use a provided variant; compose your own component from our primitives.

Unsupported and will break — `::ng-deep`, selectors targeting internal DOM structure, overriding by specificity, or depending on any token absent from the published manifest.

```

/* The pattern for every component: declare the override API, then consume
   it. Not one literal value in the file. */
:host {
  --ds-button-bg: var(--ds-color-surface-raised);
  --ds-button-fg: var(--ds-color-text);

  min-height: var(--ds-control-height);
  padding-inline: var(--ds-space-4);
  background: var(--ds-button-bg);
  color: var(--ds-button-fg);
  border-radius: var(--ds-radius-md);
}
:host([data-tone="primary"]) {
  --ds-button-bg: var(--ds-color-action);
  --ds-button-fg: var(--ds-color-text-on-action);
}
:host(:focus-visible) {
  outline: var(--ds-focus-width) solid var(--ds-color-focus);
  outline-offset: var(--ds-focus-offset);
}

```

And prefer attribute selectors on native elements — `button[dsButton]` over `<ds-button>` — so you inherit real button semantics, keyboard behaviour, form submission and type instead of re-implementing them imperfectly. Reach for a custom element name only where no native element fits: Dialog, Tabs, Toast.

Three waves, and Table is out of v1

Thirteen components across three sequenced waves, each wave going into a real screen before the next one starts. The ordering is not arbitrary — each wave is a coherent set that shares one foundation, so building its members together is what keeps their APIs agreeing with each other.

WAVE	COMPONENTS	THE SHARED FOUNDATION IT ESTABLISHES
One	Icon system, Button, Card	The whole pipeline, proven at the smallest scale. Button alone forces variants, sizes, states, focus rings, icon slots and disabled semantics; Card proves surface and elevation tokens. Icon first, because Button needs it.
Two	Field wrapper, Input, Checkbox, Radio, Switch	One <code>ControlValueAccessor</code> pattern and one error/hint/required model, shared by all five. Build them together or they will disagree — and the Field wrapper is what makes the label, hint and error contract identical everywhere.
Three	Dialog, Menu, Tooltip, Toast, Tabs, then Select / Combobox	Overlay positioning and focus management, built once on <code>@angular/cdk</code> — <code>overlay</code> and <code>a11y</code> — and reused. Keep Combobox last in the wave: it is the hardest accessibility problem in the set, and it wants Menu's foundation already working.
Out	Table / Data Grid	Post-v1. Ship a token-styled static table in the meantime — a <code>dsTable</code> directive on a plain <code><table></code> costs an afternoon and covers most needs. Sorting, virtualisation, selection and column resizing are a quarter of work.

Use the CDK without guilt. Solo, your scarcest resource is attention, and it belongs on tokens and API quality — not on a focus trap that already exists, unstyled and headless, maintained by the Angular team. The CDK is also the reason wave 3 is one wave rather than six separate problems.

THE BUILD PLAN

Your four phases, with the gate that ends each one

1

Token structure

Roughly forty tokens across colour, space, type, radius and elevation, in three tiers, with ramps generated in OKLCH. Style Dictionary builds CSS and TS in CI. Light, dark, compact and comfortable all switch at runtime in a sandbox that contains no components yet. **Gate: theme switching demonstrably works on plain divs before any component exists.**

2

Core, components, directives

The skeleton: ds-core (generated types, token re-export, CDK setup), the written conventions every later component follows, and the Stylelint rule that forbids literals. The sandbox runs `provideZonelessChangeDetection()` from this point on. Publish Button to a private registry. **Gate: it installs and renders in both an Angular 20 app and a current-version app, in CI.**

3

Token-driven components

Waves one and two. Every component ships with a documented component-token table, an axe-clean interaction test and a token-driven `:focus-visible` ring — no exceptions, no design negotiation. **Gate: one real screen rebuilt on the library.** Every gap it exposes is your actual backlog, and it will be better than any roadmap written now.



Theme provider and override example

The `dsTheme` directive, a persisted preference, and a worked example of a consumer restyling one component *legally* — through component tokens, no `!!ng-deep`. **My one remaining recommendation: split this.** The sandbox toggle belongs in phase 1, because it is what proves the token architecture — far cheaper to fix then than after thirteen components depend on it. What stays in phase 4 is the packaged directive and the documented override example.

Nothing is blocked — one thing to confirm as you read

The Angular 20 floor closed the only open question from revision 1. The single item I would still like your nod on is the phase 4 split above — pulling the runtime theme toggle forward into phase 1 and leaving the packaged directive and override example where they are. It changes the order of about two days' work and materially de-risks the token architecture.

On your go-ahead I will draft **phase 1**: the forty-token DTCC set with OKLCH ramps and the light/dark and density layers, the Style Dictionary config, and the repo scaffold — so you can run the theme switch in a sandbox before a single component exists.